SWE30011 – IoT Programming

Garden Monitor – Thomas Shanley 102588210

Summary

For my individual project I have created a plant monitoring system. It is designed to monitor the environmental conditions of a plant and display them to the user through a web application and local LED display. For this I have used a temperature + humidity sensor (DHT11) and a soil moisture sensor which record their respective data every 1 second. A 4 digit LED display is used to show the temperature, moisture and humidity data each second.

The user can cycle through each sensor by clicking a push button connected to the Arduino or by using a button on the user interface (UI). Each time the data is recorded by the sensor, it is send to a raspberry pi through a serial port in JSON format.

This data enters an express.js web server and is published to a local Maria Database on the raspberry pi for storage and later analysis. The data is also emitted through a WebSocket (Socket.JS) to the frontend, Vue.JS UI. The use of a WebSocket allows the data to reach the UI in real time and facilitates the connection between the server and client. WebSocket's use bidirectional communication, allowing the client and server to send and receive messages asynchronously as opposed to the traditional HTTP, which follows a request-response model. In addition, I have also utilised the traditional HTTP request/response model to initiate changing of the 4-digit LED display from the UI and initiate basic data analysis on the sensor data stored in the local Maria database.

Conceptual Design

Block Diagram

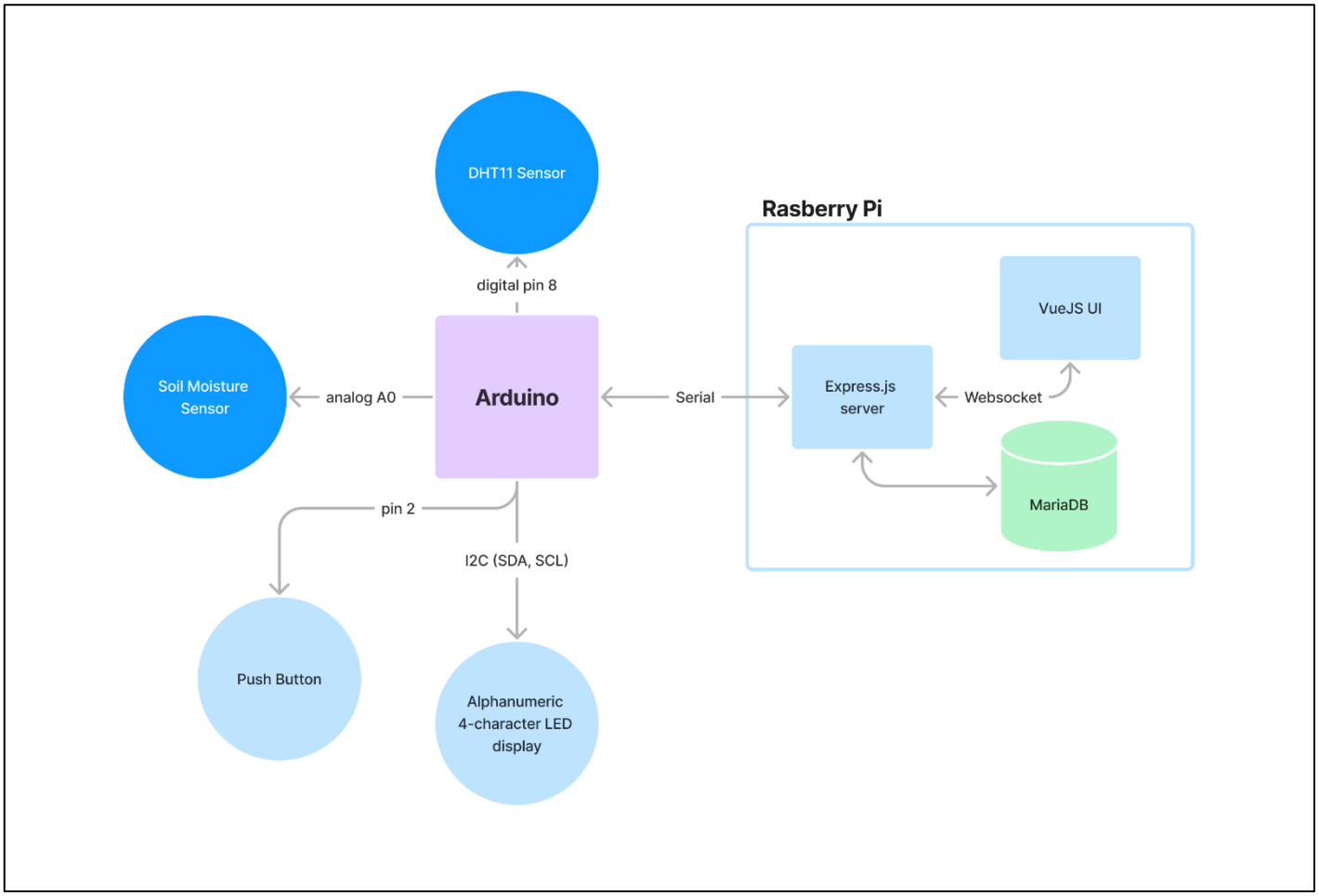

Figure 1 - Block Diagram

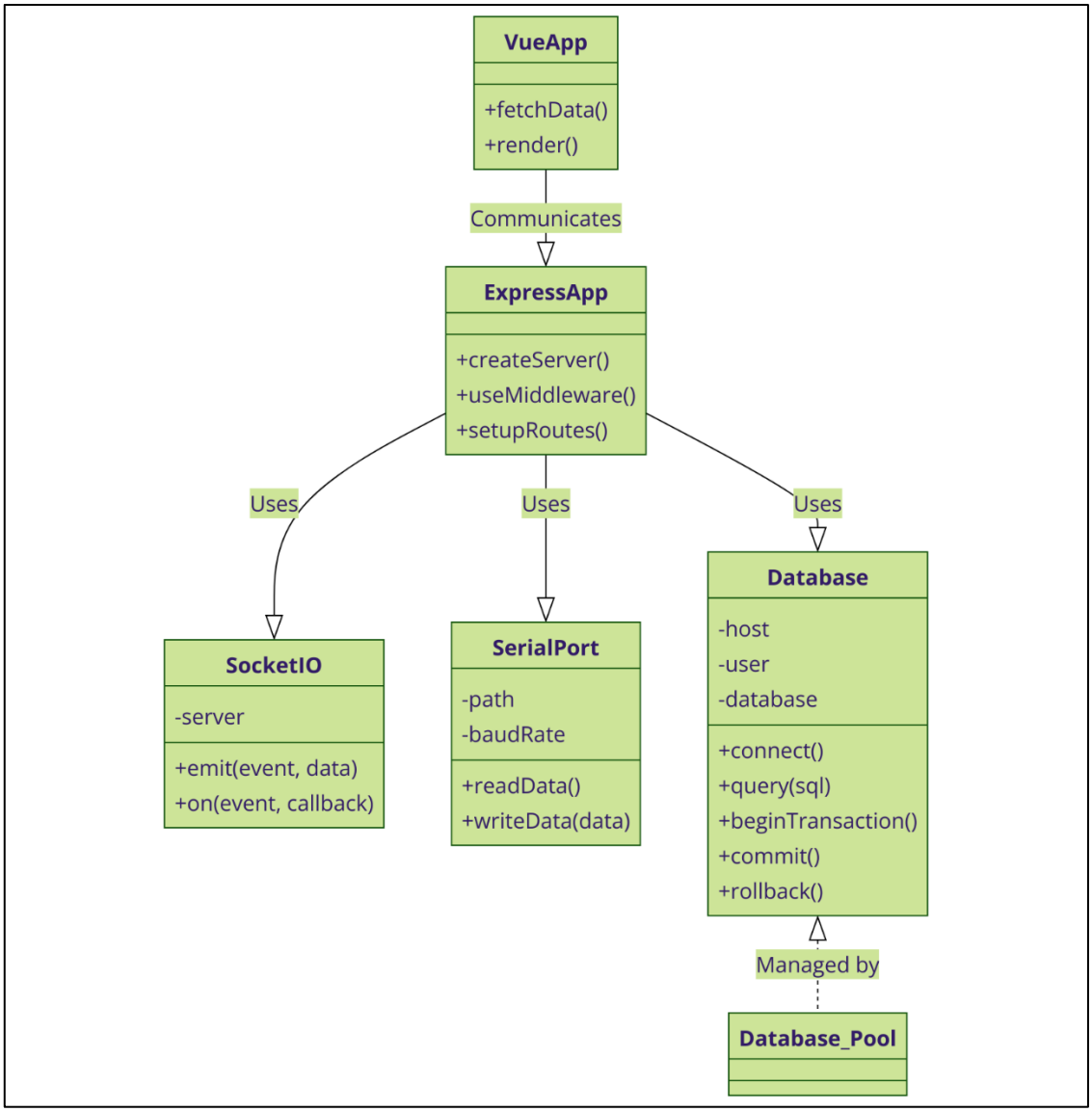

Figure 2 - UML Diagram

Implementation

Sensors

DHT11 Sensor

The DHT11 is a temperature and humidity sensor. It outputs a digital signal on the data pin 8 and requires 5 volts of power. I have utilized the DHTLib library to facilitate the integration of the sensor with the Arduino.

Soil Moisture Sensor

The soil moisture sensor measures the content of water within soil. It produces an analog output that is read on pin A0 and requires 5 volts of power.

Actuators

Alphanumeric 4-character Display

This is a LED display that can show letters and numbers in common 7 segment fashion. It is paired with a HT16K33 driver chip that is connected to I2C clock and data. This module required soldering to function.

Push Button

Standard push button. It is connected to pin 2 to allow for interrupts so that the LED display can be toggled.

Libraries

Wire library

This is a standard Arduino library for communication over the I2C/TWI interface. It is used to communication to the alphanumeric display.

Adafruit_GFX Library

This is a base library that provides a lot of written code for easy rendering of text, graphics, and images on display hardware. It serves as the foundation for other libraries like Adafruit_LEDBackpack.

Adafruit_LEDBackpack

This library is specifically designed for controlling LED matrices and segment displays with the HT16K33 driver over I2C. It simplifies controlling the Adafruit 4-character alphanumeric display.

DHT Library

This library simplifies interfacing with the DHT11 humidity and temperature sensor. It handles the digital signal timing and parsing to provide convenient methods to read temperature and humidity data.

Resources

- Adafruit (no date) Adafruit_GPS. Available at: https://github.com/adafruit/Adafruit_GPS (Accessed: 17/04/2024).
- Adafruit (no date) Adafruit_LED_Backpack. Available at: https://github.com/adafruit/Adafruit_LED_Backpack (Accessed: 17/04/2024).
- Tillaart, R. (no date) DHTlib. Available at: https://github.com/RobTillaart/DHTlib (Accessed: 17/04/2024).
- Vue.js (no date) Home. Available at: https://vuejs.org/ (Accessed: 17/04/2024).
- Adafruit (no date) HT16K33. Available at: https://github.com/adafruit/Adafruit_LED_Backpack/blob/master/examples/HT16K33/HT16K33.ino (Accessed: 17/04/2024).
- Wokwi (no date) Documentation. Available at: https://docs.wokwi.com/?utm_source=wokwi (Accessed: 17/04/2024).
- Wokwi (no date) Project details. Available at: https://wokwi.com/projects/289186888566178317 (Accessed: 17/04/2024).
- Robotics Backend (no date) Arduino Uno Pins: A Complete Practical Guide. Available at: https://roboticsbackend.com/arduino-uno-pins-a-complete-practical-guide/#Arduino_Uno_pin_diagram (Accessed: 17/04/2024).
- Jaycar Electronics (no date) Data Sheet. Available at: https://www.jaycar.com.au/medias/sys_master/images/images/10223284387870/XC3715-dataSheetMain.pdf (Accessed: 17/04/2024).
- Raspberry Tips (no date) How to Install MariaDB on a Raspberry Pi. Available at: https://raspberrytips.com/install-mariadb-raspberry-pi/ (Accessed: 17/04/2024).
- Serialport (no date) parser-readline. Available at: https://github.com/serialport/node-serialport/tree/main/packages/parser-readlin e (Accessed: 17/04/2024).

- Socket.IO (no date) Home. Available at: https://socket.io/ (Accessed: 17/04/2024).
- Socket.IO (no date) vue-client todo store. Available at: https://github.com/socketio/socket.io/blob/main/examples/basic-crud-application/vue-client/src/stores/todo.js (Accessed: 17/04/2024).
- YouTube (no date) [video title]. Available at: https://www.youtube.com/watch?v=__FSpGHx9Ow (Accessed: 17/04/2024).
- Turquoise, S. (no date) Building a Simple Web Application with Vue.js and Node.js: A Beginner's Guide. Available at: https://medium.com/@samturquoise/building-a-simple-web-application-with-vue-js-and-node-js-a-beginners-guide-66bef468118a (Accessed: 17/04/2024).

Appendix

Appendix A- Arduino Source Code

```
#include <Wire.h>

#include <Adafruit_GFX.h>
#include "Adafruit_LEDBackpack.h"
#include <dht.h>

#define DISPLAY_ADDRESS 0x70
#define DHT11_PIN 8
#define SOIL_PIN A0       // Define the pin where the DHT11 is connected
#define BUTTON_PIN 2
Adafruit_AlphaNum4 disp = Adafruit_AlphaNum4();
dht DHT;

char display_buffer[5];
long last_display_refresh = 0L;

// Current state of 7 segment display
int displayToggle = 0;

// Button state
int lastButtonState = HIGH;

int temp, moisture, humidity;

void setup() {
 Serial.begin(9600);
 pinMode(BUTTON_PIN, INPUT_PULLUP);
 disp.begin(DISPLAY_ADDRESS);
 disp.setBrightness(4);
 disp.clear();
 disp.writeDisplay();

 // Interrupt, swith display mode on button press
 attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), handleButtonPress,
RISING);
}


void loop() {
  int chk = DHT.read11(DHT11_PIN);
  temp = DHT.temperature;
  humidity = DHT.humidity;

  moisture = analogRead(SOIL_PIN);
```

```
  // Print to serial monitor (in json formatted string)
  Serial.print("sensor-data {\"temp\":");
  Serial.print(temp);
  Serial.print(", \"humidity\":");
  Serial.print(humidity);
  Serial.print(", \"moisture\":");
  Serial.print(moisture);
  Serial.println("}");

  if (Serial.available() > 0) {
    String input = Serial.readStringUntil('\n');
    if (input == "toggle-display") {
      displayToggle = (displayToggle + 1) % 3;
      Serial.println("Display toggled through serial");
    }
  }

  long current_millis = millis();
  if (current_millis - last_display_refresh > 300) {
    switch (displayToggle) {
    case 0:
      displayTemp();
      break;
    case 1:
      displayHumidity();
      break;
    case 2:
      displayMoisture();
      break;
    default:
      displayToggle = 0;
      break;
    }

    // Update the timing variable
    last_display_refresh = current_millis;
    delay(1000);
  }
}

// Interrupt service routine for button press
void handleButtonPress() {
  Serial.println("Interrupt occured");
  displayToggle = (displayToggle + 1) % 3; // Increment caseIndex and wrap
around
}

// Switches display to display temperature
```

```
void displayTemp() {
  if (isnan(temp)) {
    strcpy(display_buffer, "Err");  // Display error if reading fails
  } else {
    sprintf(display_buffer,"%3dC", temp);  // Format the temperature reading
  }
  // Send the text to the display
  for (int i = 0; i < 4; i++) {
    disp.writeDigitAscii(i, display_buffer[i]);
  }

  // Display the text
  disp.writeDisplay();
  }
void displayHumidity() {
  if (isnan(humidity)) {
    strcpy(display_buffer, "Err");  // Display error if reading fails
  } else {
    sprintf(display_buffer,"%3d%%", humidity);  // Format the temperature
reading
  }
  // Send the text to the display
  for (int i = 0; i < 4; i++) {
    disp.writeDigitAscii(i, display_buffer[i]);
  }

  // Display the text
  disp.writeDisplay();
  }
void displayMoisture() {
  if (isnan(moisture)) {
    strcpy(display_buffer, "Err");  // Display error if reading fails
  } else {
    sprintf(display_buffer,"%3d%%", moisture);  // Format the temperature
reading
  }
  // Send the text to the display
  for (int i = 0; i < 4; i++) {
    disp.writeDigitAscii(i, display_buffer[i]);
  }

  // Display the text
  disp.writeDisplay();
  }
```

Appendix B – Express Server Source code

Appendix C – Vue.JS User Interface

Both Express server and Vue Frontend are on GitHub: https://github.com/tomsshanley/SWE30011-Individual-Assignment

```
<script setup>
import { ref } from 'vue'
import {socket} from '@/socket.js'
console.log("Hello");
const temp = ref(0)
const moisture = ref(0)
const humidity = ref(0)
const tempAvg = ref(0)
const moistureAvg= ref(0)
const humidityAvg = ref(0)
socket.off()

function connect(){
  socket.connect();
}
function disconnet() {
  socket.disconnet();
}

socket.on('sensor-data', (data) => {
  if (data) {
    temp.value = data.temp
    moisture.value = data.moisture
    humidity.value = data.humidity
  }
})

async function toggleDisplay() {
  try {
    const response = await fetch('http://localhost:3000/toggle-display')
    console.log(response);
```

```
  } catch (error) {
    console.log(error);

  }
}
async function calcalateAverages() {
  try {
    const response = await (await fetch('http://localhost:3000/sensor-averages')).json()
    console.log(response);
    tempAvg.value = response[2].average
    humidityAvg.value = response[0].average
    moistureAvg.value = response[1].average
  } catch (error) {
    console.log(error);

  }
}
</script>

<template>
  <div class="container">
    <div class="temp-card">
      <h1>Temperature</h1>
      <h2>{{ temp }} C</h2>
      <h3>Average: {{ tempAvg }}</h3>
    </div>
    <div class="humidity-card">
      <h1>Humidity</h1>
      <h2>{{ humidity }} C</h2>
      <h3>Average: {{ humidityAvg }}</h3>
    </div>
```

```
    <div class="moisture-card">
      <h1>Moisture</h1>
      <h2>{{ moisture }} C</h2>
      <h3>Average: {{ moistureAvg }}</h3>
    </div>
    <!-- <button @click="connect()">Connect</button> -->
  <!-- <button @click="disconnect()">Disconnect</button> -->
  <button class="toggleDisplayButton" @click="toggleDisplay()">Toggle 7 segment display</button>
  <button class="calcalateAveragesButton" @click="calcalateAverages()">Calculate
Averages</button>
  </div>



</template>


<style scoped>
.container {
  display: flex;
  justify-content: space-around; /* This will distribute space around the cards equally */
  align-items: center; /* This will vertically align the cards in the center */
  height: 100vh; /* Sets the container height to the full viewport height */
  margin: 0 auto; /* Centers the container horizontally */
  width: 100%
}

.temp-card, .humidity-card, .moisture-card {
  flex: 1;  /* This gives each card equal width */
  margin: 10px; /* Optional: Adds some space between the cards */
  text-align: center; /* Centers the text inside each card */
}
```

```
/* Optional: Adds some basic styling to the cards */
.temp-card, .humidity-card, .moisture-card {
  border: 1px solid #ccc;
  padding: 20px;
  box-shadow: 0px 0px 10px #ccc;
}
.toggleDisplayButton {
  position: fixed; /* Keeps the button fixed during scrolling */
  top: 10px;      /* Distance from the top of the viewport */
  right: 10px;    /* Distance from the right of the viewport */
  padding: 10px 20px;
  background-color: #007BFF;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  outline: none;
}
.calcalateAveragesButton {
  position: fixed; /* Keeps the button fixed during scrolling */
  top: 10px;      /* Distance from the top of the viewport */
  left: 10px;    /* Distance from the right of the viewport */
  padding: 10px 20px;
  background-color: #007BFF;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  outline: none;
}
button:hover {
```

```
  background-color: #0056b3;
}
</style>
```